```
name="Multiplicant_B"><br>
Result: <input type="number" id="answer"
name="Multiplicant_C"><br>
<input type="button" value="Multiplication_compute_on_C++"
onclick="Multiply()">
</form>
</body>
</html>
```

Please note that in the above HTML code, *myoperations* is a class object. And *MultOfNumbers* is its public *Q_INVOKABLE* class method.

How to call the C++ methods from the Web page using the Qt-WebKit framework

Let's say, I have the following class that has the *Q_INVOKABLE* method, *MultOfNumbers*.

```
class MyJavaScriptOperations : public QObject {
Q_OBJECT
public:
Q_INVOKABLE qint32 MultOfNumbers(int a, int b) {
qDebug() << a * b;
return (a*b);
}
};
```

This class object should be added to the JavaScript window object by the following API:

```
addToJavaScriptWindowObject("name of the object", new (class
that can be accessed))
```

Here is the entire program:

```
#include <QtGui/QApplication>
#include <QApplication>
#include <QDebug>
#include <QWebFrame>
#include <QWebPage>
#include <QWebView>

class MyJavaScriptOperations : public QObject {
Q_OBJECT
public:
Q_INVOKABLE qint32 MultOfNumbers(int a, int b) {
qDebug() << a * b;
return (a*b);
}
};

int main(int argc, char *argv[])
{
QApplication a(argc, argv);
QWebView *view = new QWebView();
view->resize(400, 500);
view->page()->mainFrame()->addToJavaScriptWindowObject("myope
rations", new MyJavaScriptOperations);
view->load(QUrl("./index.html"));
view->show();
return a.exec();
}
#include "main.moc"
```

The output is given in Figure 1.

How to install a callback from C++ code to the Web page using the Qt-WebKit framework

We have already seen the call to C++ methods by JavaScript. Now, how about a callback from C++ to JavaScript? Yes, it is possible with the Qt-WebKit. There are two ways to do so. However, for the sake of neatness in design, let's discuss only the Signals and Slots mechanisms for the JavaScript callback.

Installing Signals and Slots for the JavaScript function

Here are the steps that need to be taken for the callback to be installed:

a) Add a JavaScript window object to the *javaScriptWindowObjectCleared* slot.
b) Declare a signal in the class.
c) Emit the signal.
d) In JavaScript, connect the signal to the JavaScript function slot.
   Here is the syntax to help you connect:

```
<JavaScript_window_object>.<signal_name>.connect(<JavaScript
function name>);
```

Note, you can make a callback to JavaScript only after the Web page is loaded. This can be ensured by connecting to the Slot emitted by the Signal *loadFinished()* in the C++ application.

Let's look at a real example now. This will fire a callback once the Web page is loaded.

The callback should be addressed by the JavaScript function, which will show up an alert window.

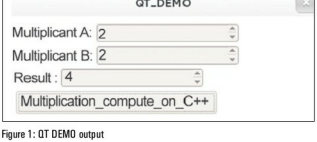

Figure 1: QT DEMO output